



TECHNICAL LEARNING FILE

DEEP-16

Computer Vision Systems

Images from representation to deployment

FORMAT	LENGTH	ACCESS
INTENSIVE FILE	50 PAGES	OFFLINE

Edition 2 - original curriculum - editorial review recommended



FILE / ORIENTATION

Purpose

01 FILE GUIDANCE

This optional advanced guide does not gate the core learning path. Apply computer vision systems with explicit assumptions, tests, tradeoffs, and limitations.

02 LOCAL USE

Index this page in the offline database and preserve its resource and page identifiers for bookmarks, notes, highlights, and progress.



FILE / ORIENTATION

Prerequisites

01 FILE GUIDANCE

Complete the related core track or pass its diagnostic.

NOUS AI ACADEMY

02 LOCAL USE

Index this page in the offline database and preserve its resource and page identifiers for bookmarks, notes, highlights, and progress.



FILE / ORIENTATION

Study Method

01 FILE GUIDANCE

For each unit, explain the concept, follow the workflow, reproduce the example, analyze a failure, and complete the applied lab. Record evidence locally.

02 LOCAL USE

Index this page in the offline database and preserve its resource and page identifiers for bookmarks, notes, highlights, and progress.



UNIT 01 / CONCEPT

Pixels and Color: Concept

01 OBJECTIVE

Explain and apply pixels and color using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Pixels and Color is a central part of computer vision systems because it changes how inputs, assumptions, implementation choices, and evidence connect. Begin by asking what problem the idea solves, what information enters the process, what result should leave it, and what evidence would show that the result is useful. In Computer Vision Systems, the terms Pixels, Color are not vocabulary to memorize in isolation; they describe a system of relationships. A strong learner can translate the idea between plain language, a diagram, a small numerical or code example, and a statement of limitations. This page establishes that translation before adding detail.

03 REASONING

Central question: When should pixels and color be used, and what observable evidence would make that choice defensible? Build a small local example that applies pixels and color, records the result, and tests one failure condition.

04 IMPLEMENTATION

```
for batch in train_loader:
    optimizer.zero_grad()
    prediction = model(batch.inputs)
    loss = objective(prediction, batch.targets)
    loss.backward()
    optimizer.step()
# validate on held-out data after the epoch
```

05 PRACTICE

1. Define Pixels and Color in one precise sentence and name one assumption.
2. Create a two-step example of pixels and color and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 01 / WORKFLOW

Pixels and Color: Workflow

01 OBJECTIVE

Explain and apply pixels and color using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Pixels and Color is a central part of computer vision systems because it changes how inputs, assumptions, implementation choices, and evidence connect. A first-principles view separates the mechanism into state, operation, constraint, and test. State describes what is currently known. The operation transforms that state. A constraint rules out invalid moves, and a test compares the output with an expectation. This structure prevents an implementation from becoming a collection of unexplained steps. It also makes debugging local: identify which state, operation, constraint, or test failed. Apply this model directly to pixels and color.

03 REASONING

Workflow: define the smallest valid input; predict the next state; perform one operation; test one invariant; then expand. Example: Build a small local example that applies pixels and color, records the result, and tests one failure condition.

04 IMPLEMENTATION

```
for batch in train_loader:
    optimizer.zero_grad()
    prediction = model(batch.inputs)
    loss = objective(prediction, batch.targets)
    loss.backward()
    optimizer.step()
# validate on held-out data after the epoch
```

05 PRACTICE

1. Define Pixels and Color in one precise sentence and name one assumption.
2. Create a two-step example of pixels and color and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 01 / WORKED EXAMPLE

Pixels and Color: Worked Example

01 OBJECTIVE

Explain and apply pixels and color using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Consider this task: Build a small local example that applies pixels and color, records the result, and tests one failure condition. First state the expected output before calculating or coding. Next choose a minimal representation and execute one step at a time. After every step, compare the intermediate state with the rule that should still hold. Finally, test a boundary case that differs from the example in exactly one important way. This worked-example pattern turns pixels and color into a repeatable method rather than a memorized answer.

03 REASONING

Worked reasoning: 1) restate the task; 2) identify knowns and unknowns; 3) choose the smallest method; 4) calculate or trace; 5) verify independently; 6) explain what would make the result fail.

04 IMPLEMENTATION

```
for batch in train_loader:
    optimizer.zero_grad()
    prediction = model(batch.inputs)
    loss = objective(prediction, batch.targets)
    loss.backward()
    optimizer.step()
# validate on held-out data after the epoch
```

05 PRACTICE

1. Define Pixels and Color in one precise sentence and name one assumption.
2. Create a two-step example of pixels and color and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 01 / FAILURE ANALYSIS

Pixels and Color: Failure Analysis

01 OBJECTIVE

Explain and apply pixels and color using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Failure analysis asks how pixels and color can produce a plausible but wrong result. Common causes include an incorrect assumption, a representation that loses information, an edge case omitted from tests, an invalid comparison, or evidence taken from the wrong stage of the process. Change one factor at a time, preserve the failing example, and write a regression test before repairing the implementation. A clear failure record is part of the learning artifact.

03 REASONING

Failure probe: construct the smallest input that breaks the naive approach to pixels and color. State the expected behavior and why the naive result is misleading.

04 IMPLEMENTATION

```
for batch in train_loader:
    optimizer.zero_grad()
    prediction = model(batch.inputs)
    loss = objective(prediction, batch.targets)
    loss.backward()
    optimizer.step()
# validate on held-out data after the epoch
```

05 PRACTICE

1. Define Pixels and Color in one precise sentence and name one assumption.
2. Create a two-step example of pixels and color and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 01 / APPLIED LAB

Pixels and Color: Applied Lab

01 OBJECTIVE

Explain and apply pixels and color using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Implementation converts the idea of pixels and color into inspectable behavior. Start with a deterministic example small enough to check by hand. Keep data preparation, core logic, and reporting separate. Add assertions for shape, type, range, or state as appropriate. Record the configuration and avoid relying on hidden global state. The goal is not merely to obtain output; it is to create an output whose origin can be explained and reproduced.

03 REASONING

Lab brief: Build a small local example that applies pixels and color, records the result, and tests one failure condition. Save the input, expected output, observed output, and a short explanation of any difference.

04 IMPLEMENTATION

```
for batch in train_loader:
    optimizer.zero_grad()
    prediction = model(batch.inputs)
    loss = objective(prediction, batch.targets)
    loss.backward()
    optimizer.step()
# validate on held-out data after the epoch
```

05 PRACTICE

1. Define Pixels and Color in one precise sentence and name one assumption.
2. Create a two-step example of pixels and color and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 02 / CONCEPT

Augmentation: Concept

01 OBJECTIVE

Explain and apply augmentation using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Augmentation is a central part of computer vision systems because it changes how inputs, assumptions, implementation choices, and evidence connect. Begin by asking what problem the idea solves, what information enters the process, what result should leave it, and what evidence would show that the result is useful. In Computer Vision Systems, the terms Augmentation are not vocabulary to memorize in isolation; they describe a system of relationships. A strong learner can translate the idea between plain language, a diagram, a small numerical or code example, and a statement of limitations. This page establishes that translation before adding detail.

03 REASONING

Central question: When should augmentation be used, and what observable evidence would make that choice defensible? Build a small local example that applies augmentation, records the result, and tests one failure condition.

04 IMPLEMENTATION

```
for batch in train_loader:
    optimizer.zero_grad()
    prediction = model(batch.inputs)
    loss = objective(prediction, batch.targets)
    loss.backward()
    optimizer.step()
# validate on held-out data after the epoch
```

05 PRACTICE

1. Define Augmentation in one precise sentence and name one assumption.
2. Create a two-step example of augmentation and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 02 / WORKFLOW

Augmentation: Workflow

01 OBJECTIVE

Explain and apply augmentation using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Augmentation is a central part of computer vision systems because it changes how inputs, assumptions, implementation choices, and evidence connect. A first-principles view separates the mechanism into state, operation, constraint, and test. State describes what is currently known. The operation transforms that state. A constraint rules out invalid moves, and a test compares the output with an expectation. This structure prevents an implementation from becoming a collection of unexplained steps. It also makes debugging local: identify which state, operation, constraint, or test failed. Apply this model directly to augmentation.

03 REASONING

Workflow: define the smallest valid input; predict the next state; perform one operation; test one invariant; then expand. Example: Build a small local example that applies augmentation, records the result, and tests one failure condition.

04 IMPLEMENTATION

```
for batch in train_loader:
    optimizer.zero_grad()
    prediction = model(batch.inputs)
    loss = objective(prediction, batch.targets)
    loss.backward()
    optimizer.step()
# validate on held-out data after the epoch
```

05 PRACTICE

1. Define Augmentation in one precise sentence and name one assumption.
2. Create a two-step example of augmentation and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 02 / WORKED EXAMPLE

Augmentation: Worked Example

01 OBJECTIVE

Explain and apply augmentation using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Consider this task: Build a small local example that applies augmentation, records the result, and tests one failure condition. First state the expected output before calculating or coding. Next choose a minimal representation and execute one step at a time. After every step, compare the intermediate state with the rule that should still hold. Finally, test a boundary case that differs from the example in exactly one important way. This worked-example pattern turns augmentation into a repeatable method rather than a memorized answer.

03 REASONING

Worked reasoning: 1) restate the task; 2) identify knowns and unknowns; 3) choose the smallest method; 4) calculate or trace; 5) verify independently; 6) explain what would make the result fail.

04 IMPLEMENTATION

```
for batch in train_loader:
    optimizer.zero_grad()
    prediction = model(batch.inputs)
    loss = objective(prediction, batch.targets)
    loss.backward()
    optimizer.step()
# validate on held-out data after the epoch
```

05 PRACTICE

1. Define Augmentation in one precise sentence and name one assumption.
2. Create a two-step example of augmentation and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 02 / FAILURE ANALYSIS

Augmentation: Failure Analysis

01 OBJECTIVE

Explain and apply augmentation using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Failure analysis asks how augmentation can produce a plausible but wrong result. Common causes include an incorrect assumption, a representation that loses information, an edge case omitted from tests, an invalid comparison, or evidence taken from the wrong stage of the process. Change one factor at a time, preserve the failing example, and write a regression test before repairing the implementation. A clear failure record is part of the learning artifact.

03 REASONING

Failure probe: construct the smallest input that breaks the naive approach to augmentation. State the expected behavior and why the naive result is misleading.

04 IMPLEMENTATION

```
for batch in train_loader:
    optimizer.zero_grad()
    prediction = model(batch.inputs)
    loss = objective(prediction, batch.targets)
    loss.backward()
    optimizer.step()
# validate on held-out data after the epoch
```

05 PRACTICE

1. Define Augmentation in one precise sentence and name one assumption.
2. Create a two-step example of augmentation and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 02 / APPLIED LAB

Augmentation: Applied Lab

01 OBJECTIVE

Explain and apply augmentation using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Implementation converts the idea of augmentation into inspectable behavior. Start with a deterministic example small enough to check by hand. Keep data preparation, core logic, and reporting separate. Add assertions for shape, type, range, or state as appropriate. Record the configuration and avoid relying on hidden global state. The goal is not merely to obtain output; it is to create an output whose origin can be explained and reproduced.

03 REASONING

Lab brief: Build a small local example that applies augmentation, records the result, and tests one failure condition. Save the input, expected output, observed output, and a short explanation of any difference.

04 IMPLEMENTATION

```
for batch in train_loader:
    optimizer.zero_grad()
    prediction = model(batch.inputs)
    loss = objective(prediction, batch.targets)
    loss.backward()
    optimizer.step()
# validate on held-out data after the epoch
```

05 PRACTICE

1. Define Augmentation in one precise sentence and name one assumption.
2. Create a two-step example of augmentation and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 03 / CONCEPT

Convolutions: Concept

01 OBJECTIVE

Explain and apply convolutions using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Convolutions is a central part of computer vision systems because it changes how inputs, assumptions, implementation choices, and evidence connect. Begin by asking what problem the idea solves, what information enters the process, what result should leave it, and what evidence would show that the result is useful. In Computer Vision Systems, the terms Convolutions are not vocabulary to memorize in isolation; they describe a system of relationships. A strong learner can translate the idea between plain language, a diagram, a small numerical or code example, and a statement of limitations. This page establishes that translation before adding detail.

03 REASONING

Central question: When should convolutions be used, and what observable evidence would make that choice defensible? Build a small local example that applies convolutions, records the result, and tests one failure condition.

04 IMPLEMENTATION

```
for batch in train_loader:
    optimizer.zero_grad()
    prediction = model(batch.inputs)
    loss = objective(prediction, batch.targets)
    loss.backward()
    optimizer.step()
# validate on held-out data after the epoch
```

05 PRACTICE

1. Define Convolutions in one precise sentence and name one assumption.
2. Create a two-step example of convolutions and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 03 / WORKFLOW

Convolutions: Workflow

01 OBJECTIVE

Explain and apply convolutions using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Convolutions is a central part of computer vision systems because it changes how inputs, assumptions, implementation choices, and evidence connect. A first-principles view separates the mechanism into state, operation, constraint, and test. State describes what is currently known. The operation transforms that state. A constraint rules out invalid moves, and a test compares the output with an expectation. This structure prevents an implementation from becoming a collection of unexplained steps. It also makes debugging local: identify which state, operation, constraint, or test failed. Apply this model directly to convolutions.

03 REASONING

Workflow: define the smallest valid input; predict the next state; perform one operation; test one invariant; then expand. Example: Build a small local example that applies convolutions, records the result, and tests one failure condition.

04 IMPLEMENTATION

```
for batch in train_loader:
    optimizer.zero_grad()
    prediction = model(batch.inputs)
    loss = objective(prediction, batch.targets)
    loss.backward()
    optimizer.step()
# validate on held-out data after the epoch
```

05 PRACTICE

1. Define Convolutions in one precise sentence and name one assumption.
2. Create a two-step example of convolutions and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 03 / WORKED EXAMPLE

Convolutions: Worked Example

01 OBJECTIVE

Explain and apply convolutions using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Consider this task: Build a small local example that applies convolutions, records the result, and tests one failure condition. First state the expected output before calculating or coding. Next choose a minimal representation and execute one step at a time. After every step, compare the intermediate state with the rule that should still hold. Finally, test a boundary case that differs from the example in exactly one important way. This worked-example pattern turns convolutions into a repeatable method rather than a memorized answer.

03 REASONING

Worked reasoning: 1) restate the task; 2) identify knowns and unknowns; 3) choose the smallest method; 4) calculate or trace; 5) verify independently; 6) explain what would make the result fail.

04 IMPLEMENTATION

```
for batch in train_loader:
    optimizer.zero_grad()
    prediction = model(batch.inputs)
    loss = objective(prediction, batch.targets)
    loss.backward()
    optimizer.step()
# validate on held-out data after the epoch
```

05 PRACTICE

1. Define Convolutions in one precise sentence and name one assumption.
2. Create a two-step example of convolutions and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 03 / FAILURE ANALYSIS

Convolutions: Failure Analysis

01 OBJECTIVE

Explain and apply convolutions using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Failure analysis asks how convolutions can produce a plausible but wrong result. Common causes include an incorrect assumption, a representation that loses information, an edge case omitted from tests, an invalid comparison, or evidence taken from the wrong stage of the process. Change one factor at a time, preserve the failing example, and write a regression test before repairing the implementation. A clear failure record is part of the learning artifact.

03 REASONING

Failure probe: construct the smallest input that breaks the naive approach to convolutions. State the expected behavior and why the naive result is misleading.

04 IMPLEMENTATION

```
for batch in train_loader:
    optimizer.zero_grad()
    prediction = model(batch.inputs)
    loss = objective(prediction, batch.targets)
    loss.backward()
    optimizer.step()
# validate on held-out data after the epoch
```

05 PRACTICE

1. Define Convolutions in one precise sentence and name one assumption.
2. Create a two-step example of convolutions and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 03 / APPLIED LAB

Convolutions: Applied Lab

01 OBJECTIVE

Explain and apply convolutions using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Implementation converts the idea of convolutions into inspectable behavior. Start with a deterministic example small enough to check by hand. Keep data preparation, core logic, and reporting separate. Add assertions for shape, type, range, or state as appropriate. Record the configuration and avoid relying on hidden global state. The goal is not merely to obtain output; it is to create an output whose origin can be explained and reproduced.

03 REASONING

Lab brief: Build a small local example that applies convolutions, records the result, and tests one failure condition. Save the input, expected output, observed output, and a short explanation of any difference.

04 IMPLEMENTATION

```
for batch in train_loader:
    optimizer.zero_grad()
    prediction = model(batch.inputs)
    loss = objective(prediction, batch.targets)
    loss.backward()
    optimizer.step()
# validate on held-out data after the epoch
```

05 PRACTICE

1. Define Convolutions in one precise sentence and name one assumption.
2. Create a two-step example of convolutions and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 04 / CONCEPT

Classification: Concept

01 OBJECTIVE

Explain and apply classification using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Classification is a central part of computer vision systems because it changes how inputs, assumptions, implementation choices, and evidence connect. Begin by asking what problem the idea solves, what information enters the process, what result should leave it, and what evidence would show that the result is useful. In Computer Vision Systems, the terms Classification are not vocabulary to memorize in isolation; they describe a system of relationships. A strong learner can translate the idea between plain language, a diagram, a small numerical or code example, and a statement of limitations. This page establishes that translation before adding detail.

03 REASONING

Central question: When should classification be used, and what observable evidence would make that choice defensible? Build a small local example that applies classification, records the result, and tests one failure condition.

04 IMPLEMENTATION

```
for batch in train_loader:
    optimizer.zero_grad()
    prediction = model(batch.inputs)
    loss = objective(prediction, batch.targets)
    loss.backward()
    optimizer.step()
# validate on held-out data after the epoch
```

05 PRACTICE

1. Define Classification in one precise sentence and name one assumption.
2. Create a two-step example of classification and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 04 / WORKFLOW

Classification: Workflow

01 OBJECTIVE

Explain and apply classification using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Classification is a central part of computer vision systems because it changes how inputs, assumptions, implementation choices, and evidence connect. A first-principles view separates the mechanism into state, operation, constraint, and test. State describes what is currently known. The operation transforms that state. A constraint rules out invalid moves, and a test compares the output with an expectation. This structure prevents an implementation from becoming a collection of unexplained steps. It also makes debugging local: identify which state, operation, constraint, or test failed. Apply this model directly to classification.

03 REASONING

Workflow: define the smallest valid input; predict the next state; perform one operation; test one invariant; then expand. Example: Build a small local example that applies classification, records the result, and tests one failure condition.

04 IMPLEMENTATION

```
for batch in train_loader:
    optimizer.zero_grad()
    prediction = model(batch.inputs)
    loss = objective(prediction, batch.targets)
    loss.backward()
    optimizer.step()
# validate on held-out data after the epoch
```

05 PRACTICE

1. Define Classification in one precise sentence and name one assumption.
2. Create a two-step example of classification and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 04 / WORKED EXAMPLE

Classification: Worked Example

01 OBJECTIVE

Explain and apply classification using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Consider this task: Build a small local example that applies classification, records the result, and tests one failure condition. First state the expected output before calculating or coding. Next choose a minimal representation and execute one step at a time. After every step, compare the intermediate state with the rule that should still hold. Finally, test a boundary case that differs from the example in exactly one important way. This worked-example pattern turns classification into a repeatable method rather than a memorized answer.

03 REASONING

Worked reasoning: 1) restate the task; 2) identify knowns and unknowns; 3) choose the smallest method; 4) calculate or trace; 5) verify independently; 6) explain what would make the result fail.

04 IMPLEMENTATION

```
for batch in train_loader:
    optimizer.zero_grad()
    prediction = model(batch.inputs)
    loss = objective(prediction, batch.targets)
    loss.backward()
    optimizer.step()
# validate on held-out data after the epoch
```

05 PRACTICE

1. Define Classification in one precise sentence and name one assumption.
2. Create a two-step example of classification and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 04 / FAILURE ANALYSIS

Classification: Failure Analysis

01 OBJECTIVE

Explain and apply classification using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Failure analysis asks how classification can produce a plausible but wrong result. Common causes include an incorrect assumption, a representation that loses information, an edge case omitted from tests, an invalid comparison, or evidence taken from the wrong stage of the process. Change one factor at a time, preserve the failing example, and write a regression test before repairing the implementation. A clear failure record is part of the learning artifact.

03 REASONING

Failure probe: construct the smallest input that breaks the naive approach to classification. State the expected behavior and why the naive result is misleading.

04 IMPLEMENTATION

```
for batch in train_loader:
    optimizer.zero_grad()
    prediction = model(batch.inputs)
    loss = objective(prediction, batch.targets)
    loss.backward()
    optimizer.step()
# validate on held-out data after the epoch
```

05 PRACTICE

1. Define Classification in one precise sentence and name one assumption.
2. Create a two-step example of classification and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 04 / APPLIED LAB

Classification: Applied Lab

01 OBJECTIVE

Explain and apply classification using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Implementation converts the idea of classification into inspectable behavior. Start with a deterministic example small enough to check by hand. Keep data preparation, core logic, and reporting separate. Add assertions for shape, type, range, or state as appropriate. Record the configuration and avoid relying on hidden global state. The goal is not merely to obtain output; it is to create an output whose origin can be explained and reproduced.

03 REASONING

Lab brief: Build a small local example that applies classification, records the result, and tests one failure condition. Save the input, expected output, observed output, and a short explanation of any difference.

04 IMPLEMENTATION

```
for batch in train_loader:
    optimizer.zero_grad()
    prediction = model(batch.inputs)
    loss = objective(prediction, batch.targets)
    loss.backward()
    optimizer.step()
# validate on held-out data after the epoch
```

05 PRACTICE

1. Define Classification in one precise sentence and name one assumption.
2. Create a two-step example of classification and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 05 / CONCEPT

Transfer Learning: Concept

01 OBJECTIVE

Explain and apply transfer learning using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Transfer Learning is a central part of computer vision systems because it changes how inputs, assumptions, implementation choices, and evidence connect. Begin by asking what problem the idea solves, what information enters the process, what result should leave it, and what evidence would show that the result is useful. In Computer Vision Systems, the terms Transfer, Learning are not vocabulary to memorize in isolation; they describe a system of relationships. A strong learner can translate the idea between plain language, a diagram, a small numerical or code example, and a statement of limitations. This page establishes that translation before adding detail.

03 REASONING

Central question: When should transfer learning be used, and what observable evidence would make that choice defensible? Build a small local example that applies transfer learning, records the result, and tests one failure condition.

04 IMPLEMENTATION

```
for batch in train_loader:
    optimizer.zero_grad()
    prediction = model(batch.inputs)
    loss = objective(prediction, batch.targets)
    loss.backward()
    optimizer.step()
# validate on held-out data after the epoch
```

05 PRACTICE

1. Define Transfer Learning in one precise sentence and name one assumption.
2. Create a two-step example of transfer learning and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 05 / WORKFLOW

Transfer Learning: Workflow

01 OBJECTIVE

Explain and apply transfer learning using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Transfer Learning is a central part of computer vision systems because it changes how inputs, assumptions, implementation choices, and evidence connect. A first-principles view separates the mechanism into state, operation, constraint, and test. State describes what is currently known. The operation transforms that state. A constraint rules out invalid moves, and a test compares the output with an expectation. This structure prevents an implementation from becoming a collection of unexplained steps. It also makes debugging local: identify which state, operation, constraint, or test failed. Apply this model directly to transfer learning.

03 REASONING

Workflow: define the smallest valid input; predict the next state; perform one operation; test one invariant; then expand. Example: Build a small local example that applies transfer learning, records the result, and tests one failure condition.

04 IMPLEMENTATION

```
for batch in train_loader:
    optimizer.zero_grad()
    prediction = model(batch.inputs)
    loss = objective(prediction, batch.targets)
    loss.backward()
    optimizer.step()
# validate on held-out data after the epoch
```

05 PRACTICE

1. Define Transfer Learning in one precise sentence and name one assumption.
2. Create a two-step example of transfer learning and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 05 / WORKED EXAMPLE

Transfer Learning: Worked Example

01 OBJECTIVE

Explain and apply transfer learning using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Consider this task: Build a small local example that applies transfer learning, records the result, and tests one failure condition. First state the expected output before calculating or coding. Next choose a minimal representation and execute one step at a time. After every step, compare the intermediate state with the rule that should still hold. Finally, test a boundary case that differs from the example in exactly one important way. This worked-example pattern turns transfer learning into a repeatable method rather than a memorized answer.

03 REASONING

Worked reasoning: 1) restate the task; 2) identify knowns and unknowns; 3) choose the smallest method; 4) calculate or trace; 5) verify independently; 6) explain what would make the result fail.

04 IMPLEMENTATION

```
for batch in train_loader:
    optimizer.zero_grad()
    prediction = model(batch.inputs)
    loss = objective(prediction, batch.targets)
    loss.backward()
    optimizer.step()
# validate on held-out data after the epoch
```

05 PRACTICE

1. Define Transfer Learning in one precise sentence and name one assumption.
2. Create a two-step example of transfer learning and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 05 / FAILURE ANALYSIS

Transfer Learning: Failure Analysis

01 OBJECTIVE

Explain and apply transfer learning using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Failure analysis asks how transfer learning can produce a plausible but wrong result. Common causes include an incorrect assumption, a representation that loses information, an edge case omitted from tests, an invalid comparison, or evidence taken from the wrong stage of the process. Change one factor at a time, preserve the failing example, and write a regression test before repairing the implementation. A clear failure record is part of the learning artifact.

03 REASONING

Failure probe: construct the smallest input that breaks the naive approach to transfer learning. State the expected behavior and why the naive result is misleading.

04 IMPLEMENTATION

```
for batch in train_loader:
    optimizer.zero_grad()
    prediction = model(batch.inputs)
    loss = objective(prediction, batch.targets)
    loss.backward()
    optimizer.step()
# validate on held-out data after the epoch
```

05 PRACTICE

1. Define Transfer Learning in one precise sentence and name one assumption.
2. Create a two-step example of transfer learning and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 05 / APPLIED LAB

Transfer Learning: Applied Lab

01 OBJECTIVE

Explain and apply transfer learning using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Implementation converts the idea of transfer learning into inspectable behavior. Start with a deterministic example small enough to check by hand. Keep data preparation, core logic, and reporting separate. Add assertions for shape, type, range, or state as appropriate. Record the configuration and avoid relying on hidden global state. The goal is not merely to obtain output; it is to create an output whose origin can be explained and reproduced.

03 REASONING

Lab brief: Build a small local example that applies transfer learning, records the result, and tests one failure condition. Save the input, expected output, observed output, and a short explanation of any difference.

04 IMPLEMENTATION

```
for batch in train_loader:
    optimizer.zero_grad()
    prediction = model(batch.inputs)
    loss = objective(prediction, batch.targets)
    loss.backward()
    optimizer.step()
# validate on held-out data after the epoch
```

05 PRACTICE

1. Define Transfer Learning in one precise sentence and name one assumption.
2. Create a two-step example of transfer learning and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 06 / CONCEPT

Detection: Concept

01 OBJECTIVE

Explain and apply detection using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Detection is a central part of computer vision systems because it changes how inputs, assumptions, implementation choices, and evidence connect. Begin by asking what problem the idea solves, what information enters the process, what result should leave it, and what evidence would show that the result is useful. In Computer Vision Systems, the terms Detection are not vocabulary to memorize in isolation; they describe a system of relationships. A strong learner can translate the idea between plain language, a diagram, a small numerical or code example, and a statement of limitations. This page establishes that translation before adding detail.

03 REASONING

Central question: When should detection be used, and what observable evidence would make that choice defensible? Build a small local example that applies detection, records the result, and tests one failure condition.

04 IMPLEMENTATION

```
for batch in train_loader:
    optimizer.zero_grad()
    prediction = model(batch.inputs)
    loss = objective(prediction, batch.targets)
    loss.backward()
    optimizer.step()
# validate on held-out data after the epoch
```

05 PRACTICE

1. Define Detection in one precise sentence and name one assumption.
2. Create a two-step example of detection and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 06 / WORKFLOW

Detection: Workflow

01 OBJECTIVE

Explain and apply detection using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Detection is a central part of computer vision systems because it changes how inputs, assumptions, implementation choices, and evidence connect. A first-principles view separates the mechanism into state, operation, constraint, and test. State describes what is currently known. The operation transforms that state. A constraint rules out invalid moves, and a test compares the output with an expectation. This structure prevents an implementation from becoming a collection of unexplained steps. It also makes debugging local: identify which state, operation, constraint, or test failed. Apply this model directly to detection.

03 REASONING

Workflow: define the smallest valid input; predict the next state; perform one operation; test one invariant; then expand. Example: Build a small local example that applies detection, records the result, and tests one failure condition.

04 IMPLEMENTATION

```
for batch in train_loader:
    optimizer.zero_grad()
    prediction = model(batch.inputs)
    loss = objective(prediction, batch.targets)
    loss.backward()
    optimizer.step()
# validate on held-out data after the epoch
```

05 PRACTICE

1. Define Detection in one precise sentence and name one assumption.
2. Create a two-step example of detection and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 06 / WORKED EXAMPLE

Detection: Worked Example

01 OBJECTIVE

Explain and apply detection using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Consider this task: Build a small local example that applies detection, records the result, and tests one failure condition. First state the expected output before calculating or coding. Next choose a minimal representation and execute one step at a time. After every step, compare the intermediate state with the rule that should still hold. Finally, test a boundary case that differs from the example in exactly one important way. This worked-example pattern turns detection into a repeatable method rather than a memorized answer.

03 REASONING

Worked reasoning: 1) restate the task; 2) identify knowns and unknowns; 3) choose the smallest method; 4) calculate or trace; 5) verify independently; 6) explain what would make the result fail.

04 IMPLEMENTATION

```
for batch in train_loader:
    optimizer.zero_grad()
    prediction = model(batch.inputs)
    loss = objective(prediction, batch.targets)
    loss.backward()
    optimizer.step()
# validate on held-out data after the epoch
```

05 PRACTICE

1. Define Detection in one precise sentence and name one assumption.
2. Create a two-step example of detection and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 06 / FAILURE ANALYSIS

Detection: Failure Analysis

01 OBJECTIVE

Explain and apply detection using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Failure analysis asks how detection can produce a plausible but wrong result. Common causes include an incorrect assumption, a representation that loses information, an edge case omitted from tests, an invalid comparison, or evidence taken from the wrong stage of the process. Change one factor at a time, preserve the failing example, and write a regression test before repairing the implementation. A clear failure record is part of the learning artifact.

03 REASONING

Failure probe: construct the smallest input that breaks the naive approach to detection. State the expected behavior and why the naive result is misleading.

04 IMPLEMENTATION

```
for batch in train_loader:
    optimizer.zero_grad()
    prediction = model(batch.inputs)
    loss = objective(prediction, batch.targets)
    loss.backward()
    optimizer.step()
# validate on held-out data after the epoch
```

05 PRACTICE

1. Define Detection in one precise sentence and name one assumption.
2. Create a two-step example of detection and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 06 / APPLIED LAB

Detection: Applied Lab

01 OBJECTIVE

Explain and apply detection using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Implementation converts the idea of detection into inspectable behavior. Start with a deterministic example small enough to check by hand. Keep data preparation, core logic, and reporting separate. Add assertions for shape, type, range, or state as appropriate. Record the configuration and avoid relying on hidden global state. The goal is not merely to obtain output; it is to create an output whose origin can be explained and reproduced.

03 REASONING

Lab brief: Build a small local example that applies detection, records the result, and tests one failure condition. Save the input, expected output, observed output, and a short explanation of any difference.

04 IMPLEMENTATION

```
for batch in train_loader:
    optimizer.zero_grad()
    prediction = model(batch.inputs)
    loss = objective(prediction, batch.targets)
    loss.backward()
    optimizer.step()
# validate on held-out data after the epoch
```

05 PRACTICE

1. Define Detection in one precise sentence and name one assumption.
2. Create a two-step example of detection and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 07 / CONCEPT

Segmentation: Concept

01 OBJECTIVE

Explain and apply segmentation using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Segmentation is a central part of computer vision systems because it changes how inputs, assumptions, implementation choices, and evidence connect. Begin by asking what problem the idea solves, what information enters the process, what result should leave it, and what evidence would show that the result is useful. In Computer Vision Systems, the terms Segmentation are not vocabulary to memorize in isolation; they describe a system of relationships. A strong learner can translate the idea between plain language, a diagram, a small numerical or code example, and a statement of limitations. This page establishes that translation before adding detail.

03 REASONING

Central question: When should segmentation be used, and what observable evidence would make that choice defensible? Build a small local example that applies segmentation, records the result, and tests one failure condition.

04 IMPLEMENTATION

```
for batch in train_loader:
    optimizer.zero_grad()
    prediction = model(batch.inputs)
    loss = objective(prediction, batch.targets)
    loss.backward()
    optimizer.step()
# validate on held-out data after the epoch
```

05 PRACTICE

1. Define Segmentation in one precise sentence and name one assumption.
2. Create a two-step example of segmentation and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 07 / WORKFLOW

Segmentation: Workflow

01 OBJECTIVE

Explain and apply segmentation using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Segmentation is a central part of computer vision systems because it changes how inputs, assumptions, implementation choices, and evidence connect. A first-principles view separates the mechanism into state, operation, constraint, and test. State describes what is currently known. The operation transforms that state. A constraint rules out invalid moves, and a test compares the output with an expectation. This structure prevents an implementation from becoming a collection of unexplained steps. It also makes debugging local: identify which state, operation, constraint, or test failed. Apply this model directly to segmentation.

03 REASONING

Workflow: define the smallest valid input; predict the next state; perform one operation; test one invariant; then expand. Example: Build a small local example that applies segmentation, records the result, and tests one failure condition.

04 IMPLEMENTATION

```
for batch in train_loader:
    optimizer.zero_grad()
    prediction = model(batch.inputs)
    loss = objective(prediction, batch.targets)
    loss.backward()
    optimizer.step()
# validate on held-out data after the epoch
```

05 PRACTICE

1. Define Segmentation in one precise sentence and name one assumption.
2. Create a two-step example of segmentation and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 07 / WORKED EXAMPLE

Segmentation: Worked Example

01 OBJECTIVE

Explain and apply segmentation using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Consider this task: Build a small local example that applies segmentation, records the result, and tests one failure condition. First state the expected output before calculating or coding. Next choose a minimal representation and execute one step at a time. After every step, compare the intermediate state with the rule that should still hold. Finally, test a boundary case that differs from the example in exactly one important way. This worked-example pattern turns segmentation into a repeatable method rather than a memorized answer.

03 REASONING

Worked reasoning: 1) restate the task; 2) identify knowns and unknowns; 3) choose the smallest method; 4) calculate or trace; 5) verify independently; 6) explain what would make the result fail.

04 IMPLEMENTATION

```
for batch in train_loader:
    optimizer.zero_grad()
    prediction = model(batch.inputs)
    loss = objective(prediction, batch.targets)
    loss.backward()
    optimizer.step()
# validate on held-out data after the epoch
```

05 PRACTICE

1. Define Segmentation in one precise sentence and name one assumption.
2. Create a two-step example of segmentation and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 07 / FAILURE ANALYSIS

Segmentation: Failure Analysis

01 OBJECTIVE

Explain and apply segmentation using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Failure analysis asks how segmentation can produce a plausible but wrong result. Common causes include an incorrect assumption, a representation that loses information, an edge case omitted from tests, an invalid comparison, or evidence taken from the wrong stage of the process. Change one factor at a time, preserve the failing example, and write a regression test before repairing the implementation. A clear failure record is part of the learning artifact.

03 REASONING

Failure probe: construct the smallest input that breaks the naive approach to segmentation. State the expected behavior and why the naive result is misleading.

04 IMPLEMENTATION

```
for batch in train_loader:
    optimizer.zero_grad()
    prediction = model(batch.inputs)
    loss = objective(prediction, batch.targets)
    loss.backward()
    optimizer.step()
# validate on held-out data after the epoch
```

05 PRACTICE

1. Define Segmentation in one precise sentence and name one assumption.
2. Create a two-step example of segmentation and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 07 / APPLIED LAB

Segmentation: Applied Lab

01 OBJECTIVE

Explain and apply segmentation using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Implementation converts the idea of segmentation into inspectable behavior. Start with a deterministic example small enough to check by hand. Keep data preparation, core logic, and reporting separate. Add assertions for shape, type, range, or state as appropriate. Record the configuration and avoid relying on hidden global state. The goal is not merely to obtain output; it is to create an output whose origin can be explained and reproduced.

03 REASONING

Lab brief: Build a small local example that applies segmentation, records the result, and tests one failure condition. Save the input, expected output, observed output, and a short explanation of any difference.

04 IMPLEMENTATION

```
for batch in train_loader:
    optimizer.zero_grad()
    prediction = model(batch.inputs)
    loss = objective(prediction, batch.targets)
    loss.backward()
    optimizer.step()
# validate on held-out data after the epoch
```

05 PRACTICE

1. Define Segmentation in one precise sentence and name one assumption.
2. Create a two-step example of segmentation and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 08 / CONCEPT

Evaluation: Concept

01 OBJECTIVE

Explain and apply evaluation using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Evaluation is a central part of computer vision systems because it changes how inputs, assumptions, implementation choices, and evidence connect. Begin by asking what problem the idea solves, what information enters the process, what result should leave it, and what evidence would show that the result is useful. In Computer Vision Systems, the terms Evaluation are not vocabulary to memorize in isolation; they describe a system of relationships. A strong learner can translate the idea between plain language, a diagram, a small numerical or code example, and a statement of limitations. This page establishes that translation before adding detail.

03 REASONING

Central question: When should evaluation be used, and what observable evidence would make that choice defensible? Build a small local example that applies evaluation, records the result, and tests one failure condition.

04 IMPLEMENTATION

```
for batch in train_loader:
    optimizer.zero_grad()
    prediction = model(batch.inputs)
    loss = objective(prediction, batch.targets)
    loss.backward()
    optimizer.step()
# validate on held-out data after the epoch
```

05 PRACTICE

1. Define Evaluation in one precise sentence and name one assumption.
2. Create a two-step example of evaluation and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 08 / WORKFLOW

Evaluation: Workflow

01 OBJECTIVE

Explain and apply evaluation using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Evaluation is a central part of computer vision systems because it changes how inputs, assumptions, implementation choices, and evidence connect. A first-principles view separates the mechanism into state, operation, constraint, and test. State describes what is currently known. The operation transforms that state. A constraint rules out invalid moves, and a test compares the output with an expectation. This structure prevents an implementation from becoming a collection of unexplained steps. It also makes debugging local: identify which state, operation, constraint, or test failed. Apply this model directly to evaluation.

03 REASONING

Workflow: define the smallest valid input; predict the next state; perform one operation; test one invariant; then expand. Example: Build a small local example that applies evaluation, records the result, and tests one failure condition.

04 IMPLEMENTATION

```
for batch in train_loader:
    optimizer.zero_grad()
    prediction = model(batch.inputs)
    loss = objective(prediction, batch.targets)
    loss.backward()
    optimizer.step()
# validate on held-out data after the epoch
```

05 PRACTICE

1. Define Evaluation in one precise sentence and name one assumption.
2. Create a two-step example of evaluation and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 08 / WORKED EXAMPLE

Evaluation: Worked Example

01 OBJECTIVE

Explain and apply evaluation using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Consider this task: Build a small local example that applies evaluation, records the result, and tests one failure condition. First state the expected output before calculating or coding. Next choose a minimal representation and execute one step at a time. After every step, compare the intermediate state with the rule that should still hold. Finally, test a boundary case that differs from the example in exactly one important way. This worked-example pattern turns evaluation into a repeatable method rather than a memorized answer.

03 REASONING

Worked reasoning: 1) restate the task; 2) identify knowns and unknowns; 3) choose the smallest method; 4) calculate or trace; 5) verify independently; 6) explain what would make the result fail.

04 IMPLEMENTATION

```
for batch in train_loader:
    optimizer.zero_grad()
    prediction = model(batch.inputs)
    loss = objective(prediction, batch.targets)
    loss.backward()
    optimizer.step()
# validate on held-out data after the epoch
```

05 PRACTICE

1. Define Evaluation in one precise sentence and name one assumption.
2. Create a two-step example of evaluation and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 08 / FAILURE ANALYSIS

Evaluation: Failure Analysis

01 OBJECTIVE

Explain and apply evaluation using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Failure analysis asks how evaluation can produce a plausible but wrong result. Common causes include an incorrect assumption, a representation that loses information, an edge case omitted from tests, an invalid comparison, or evidence taken from the wrong stage of the process. Change one factor at a time, preserve the failing example, and write a regression test before repairing the implementation. A clear failure record is part of the learning artifact.

03 REASONING

Failure probe: construct the smallest input that breaks the naive approach to evaluation. State the expected behavior and why the naive result is misleading.

04 IMPLEMENTATION

```
for batch in train_loader:
    optimizer.zero_grad()
    prediction = model(batch.inputs)
    loss = objective(prediction, batch.targets)
    loss.backward()
    optimizer.step()
# validate on held-out data after the epoch
```

05 PRACTICE

1. Define Evaluation in one precise sentence and name one assumption.
2. Create a two-step example of evaluation and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 08 / APPLIED LAB

Evaluation: Applied Lab

01 OBJECTIVE

Explain and apply evaluation using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Implementation converts the idea of evaluation into inspectable behavior. Start with a deterministic example small enough to check by hand. Keep data preparation, core logic, and reporting separate. Add assertions for shape, type, range, or state as appropriate. Record the configuration and avoid relying on hidden global state. The goal is not merely to obtain output; it is to create an output whose origin can be explained and reproduced.

03 REASONING

Lab brief: Build a small local example that applies evaluation, records the result, and tests one failure condition. Save the input, expected output, observed output, and a short explanation of any difference.

04 IMPLEMENTATION

```
for batch in train_loader:
    optimizer.zero_grad()
    prediction = model(batch.inputs)
    loss = objective(prediction, batch.targets)
    loss.backward()
    optimizer.step()
# validate on held-out data after the epoch
```

05 PRACTICE

1. Define Evaluation in one precise sentence and name one assumption.
2. Create a two-step example of evaluation and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 09 / CONCEPT

Deployment: Concept

01 OBJECTIVE

Explain and apply deployment using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Deployment is a central part of computer vision systems because it changes how inputs, assumptions, implementation choices, and evidence connect. Begin by asking what problem the idea solves, what information enters the process, what result should leave it, and what evidence would show that the result is useful. In Computer Vision Systems, the terms Deployment are not vocabulary to memorize in isolation; they describe a system of relationships. A strong learner can translate the idea between plain language, a diagram, a small numerical or code example, and a statement of limitations. This page establishes that translation before adding detail.

03 REASONING

Central question: When should deployment be used, and what observable evidence would make that choice defensible? Build a small local example that applies deployment, records the result, and tests one failure condition.

04 IMPLEMENTATION

```
for batch in train_loader:
    optimizer.zero_grad()
    prediction = model(batch.inputs)
    loss = objective(prediction, batch.targets)
    loss.backward()
    optimizer.step()
# validate on held-out data after the epoch
```

05 PRACTICE

1. Define Deployment in one precise sentence and name one assumption.
2. Create a two-step example of deployment and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 09 / WORKFLOW

Deployment: Workflow

01 OBJECTIVE

Explain and apply deployment using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Deployment is a central part of computer vision systems because it changes how inputs, assumptions, implementation choices, and evidence connect. A first-principles view separates the mechanism into state, operation, constraint, and test. State describes what is currently known. The operation transforms that state. A constraint rules out invalid moves, and a test compares the output with an expectation. This structure prevents an implementation from becoming a collection of unexplained steps. It also makes debugging local: identify which state, operation, constraint, or test failed. Apply this model directly to deployment.

03 REASONING

Workflow: define the smallest valid input; predict the next state; perform one operation; test one invariant; then expand. Example: Build a small local example that applies deployment, records the result, and tests one failure condition.

04 IMPLEMENTATION

```
for batch in train_loader:
    optimizer.zero_grad()
    prediction = model(batch.inputs)
    loss = objective(prediction, batch.targets)
    loss.backward()
    optimizer.step()
# validate on held-out data after the epoch
```

05 PRACTICE

1. Define Deployment in one precise sentence and name one assumption.
2. Create a two-step example of deployment and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 09 / WORKED EXAMPLE

Deployment: Worked Example

01 OBJECTIVE

Explain and apply deployment using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Consider this task: Build a small local example that applies deployment, records the result, and tests one failure condition. First state the expected output before calculating or coding. Next choose a minimal representation and execute one step at a time. After every step, compare the intermediate state with the rule that should still hold. Finally, test a boundary case that differs from the example in exactly one important way. This worked-example pattern turns deployment into a repeatable method rather than a memorized answer.

03 REASONING

Worked reasoning: 1) restate the task; 2) identify knowns and unknowns; 3) choose the smallest method; 4) calculate or trace; 5) verify independently; 6) explain what would make the result fail.

04 IMPLEMENTATION

```
for batch in train_loader:
    optimizer.zero_grad()
    prediction = model(batch.inputs)
    loss = objective(prediction, batch.targets)
    loss.backward()
    optimizer.step()
# validate on held-out data after the epoch
```

05 PRACTICE

1. Define Deployment in one precise sentence and name one assumption.
2. Create a two-step example of deployment and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 09 / FAILURE ANALYSIS

Deployment: Failure Analysis

01 OBJECTIVE

Explain and apply deployment using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Failure analysis asks how deployment can produce a plausible but wrong result. Common causes include an incorrect assumption, a representation that loses information, an edge case omitted from tests, an invalid comparison, or evidence taken from the wrong stage of the process. Change one factor at a time, preserve the failing example, and write a regression test before repairing the implementation. A clear failure record is part of the learning artifact.

03 REASONING

Failure probe: construct the smallest input that breaks the naive approach to deployment. State the expected behavior and why the naive result is misleading.

04 IMPLEMENTATION

```
for batch in train_loader:
    optimizer.zero_grad()
    prediction = model(batch.inputs)
    loss = objective(prediction, batch.targets)
    loss.backward()
    optimizer.step()
# validate on held-out data after the epoch
```

05 PRACTICE

1. Define Deployment in one precise sentence and name one assumption.
2. Create a two-step example of deployment and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



UNIT 09 / APPLIED LAB

Deployment: Applied Lab

01 OBJECTIVE

Explain and apply deployment using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Implementation converts the idea of deployment into inspectable behavior. Start with a deterministic example small enough to check by hand. Keep data preparation, core logic, and reporting separate. Add assertions for shape, type, range, or state as appropriate. Record the configuration and avoid relying on hidden global state. The goal is not merely to obtain output; it is to create an output whose origin can be explained and reproduced.

03 REASONING

Lab brief: Build a small local example that applies deployment, records the result, and tests one failure condition. Save the input, expected output, observed output, and a short explanation of any difference.

04 IMPLEMENTATION

```
for batch in train_loader:
    optimizer.zero_grad()
    prediction = model(batch.inputs)
    loss = objective(prediction, batch.targets)
    loss.backward()
    optimizer.step()
# validate on held-out data after the epoch
```

05 PRACTICE

1. Define Deployment in one precise sentence and name one assumption.
2. Create a two-step example of deployment and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.



FILE / ORIENTATION

Deep-Dive Capstone

01 FILE GUIDANCE

Build a compact, reproducible artifact demonstrating computer vision systems. Include assumptions, a baseline, tests, failure analysis, results, limitations, and instructions for repeating the work offline.

02 LOCAL USE

Index this page in the offline database and preserve its resource and page identifiers for bookmarks, notes, highlights, and progress.